

Modules

Menu Launcher

Master's in Computer Security Engineering Dynamic Programming Languages - Menu Launcher

Author: Martinho Caeiro (23917)

Description:

Simple menu launcher to manage/run scripts in the ./scripts folder.

Usage:

```
python3 menu_launcher.py
```

Example

```
python3 menu_launcher.py
```

```
menu_launcher.clear_screen()
```

```
menu_launcher.ensure_dirs()
```

```
menu_launcher.main_menu()
```

```
menu_launcher.prompt_args(arg_defs)
```

```
menu_launcher.run_script(path, args)
```

```
menu_launcher.show_menu()
```

Scripts Configuration

Master's in Computer Security Engineering Dynamic Programming Languages - Scripts Configuration

Author: Martinho Caeiro (23917)

Description:

Defines the configuration of available scripts, including names, files, and required arguments.

Log Analyzer

Master's in Computer Security Engineering Dynamic Programming Languages - Log Analyzer

Author: Martinho Caeiro (23917)

Description:

Analyze Apache/Nginx access logs and OpenSSH auth logs. Extracts IP, timestamp, service, status, and details. Optionally resolves country via GeoLite2 database and generates JSON/CSV/PDF reports.

Usage:

```
python3 log_analyzer.py -files LOG_FILE ... -outdir OUTPUT_DIR [-geoip-db MMDB_PATH]
```

Example

```
python3 log_analyzer.py -files access.log auth.log -outdir reports python3 log_analyzer.py -files access.log -outdir reports -geoip-db data/GeoLite2-City.mmdb
```

Formats:

- Apache/Nginx combined log format
- OpenSSH auth.log syslog format

- UFW (Uncomplicated Firewall) log format

`class log_analyzer.GeoIPResolver(db_path: str | None = None)`

Bases: **object**

country_for(ip: str) → str

`log_analyzer.analyze_files(file_paths: List[str], geoip_db: str | None, outdir: str) → List[Dict]`

`log_analyzer.create_pdf_report(events: List[Dict], pdf_path: str, title: str)`

Generate a PDF report from events using reportlab.

`log_analyzer.find_default_geoip_db() → str | None`

Look for GeoLite2 DB files inside the repository `src/data` folder.

Checks, in order: - `src/data/GeoLite2-City.mmdb` - `src/data/GeoLite2-Country.mmdb` Returns the first existing absolute path, or None.

`log_analyzer.init_optional_deps()`

Load optional dependencies only when executing the script.

`log_analyzer.main(argv=None)`

`log_analyzer.parse_apache_line(line: str) → Dict | None`

`log_analyzer.parse_ssh_line(line: str) → Dict | None`

`log_analyzer.parse_ufw_line(line: str) → Dict | None`

`log_analyzer.summarize_events(events)`

Print a small summary: counts per service and top countries.

Package Manager

Shared package dependency manager.

Provides a centralized function for ensuring required packages are installed.

`package_manager.ensure_package(package_name, import_name=None, prompt=True)`

Ensure a package is installed, optionally prompting for installation.

- Parâmetros:**
- **package_name** – The package name for pip install
 - **import_name** – The module name for import (if different from `package_name`)
 - **prompt** – Whether to prompt user for installation if missing

Retorno: The imported module on success, or None on failure/decline.

Password Manager

Master's in Computer Security Engineering Dynamic Programming Languages - Password Manager

Author: Martinho Caeiro (23917)

Description:

Simple password manager with RSA + Fernet encryption and TOTP 2FA. Supports interactive mode and command-line operations.

Usage:

`python3 password_manager.py [-init] [--create|--list|--view|--update|--delete] [OPTIONS]`

Example

```
python3 password_manager.py --init python3 password_manager.py --create --url example.com --user
alice --pass s3cr3t python3 password_manager.py --list --otp 123456 python3 password_manager.py (inte-
ractive mode if no arguments)
```

Security:

- RSA 2048-bit encryption for key storage
- Fernet (AES-128) encryption for records
- TOTP 2FA authentication for sensitive operations
- All files stored in src/data/ directory

Nota:

Educational implementation. Do not use for production secrets.

`password_manager.check_and_auto_init()`

Check if initialization is needed and perform it automatically.

`password_manager.cmd_create(args)`

`password_manager.cmd_delete(args)`

`password_manager.cmd_list(args)`

`password_manager.cmd_update(args)`

`password_manager.cmd_view(args)`

`password_manager.decrypt_record(enc_cipher_b64, enc_key_b64)`

`password_manager.encrypt_record_plaintext(plaintext_bytes)`

Encrypt plaintext bytes using Fernet and encrypt the key with RSA public key.

`password_manager.ensure_files_dir()`

`password_manager.ensure_runtime_deps()`

Ensure optional dependencies are available when running the script.

`password_manager.init_keys_and_totp()`

Generate RSA keypair and setup TOTP 2FA with QR code.

`password_manager.load_private_key()`

`password_manager.load_public_key()`

`password_manager.load_records()`

`password_manager.main()`

`password_manager.parse_args()`

`password_manager.require_otp(code)`

Verify TOTP code for 2FA authentication.

Parâmetros: `code` – The 6-digit TOTP code from the authenticator app

Retorno: True if code is valid, False otherwise

`password_manager.run_interactive_menu()`

`password_manager.save_records(recs)`

Port Scanner

Master's in Computer Security Engineering Dynamic Programming Languages - Port Scanner

Author: Martinho Caeiro (23917)

Description:

Scans ports on multiple targets to identify which are open.

Usage:

```
python3 port_scanner.py <targets> <start_port> <end_port> [-timeout SECONDS] [-output FILE]
```

Example

```
python3 port_scanner.py 127.0.0.1 1 1024 python3 port_scanner.py target1.com,target2.com 1 65535 --output report.json
```

Nota:

Targets can be comma-separated (host1.com,host2.com,192.168.1.1)

```
port_scanner.main()
```

```
port_scanner.scan_host(host, start_port, end_port, timeout=2)
```

Scan ports on a specific host.

Port Knocker

Master's in Computer Security Engineering Dynamic Programming Languages - Port Knocker

Author: Martinho Caeiro (23917)

Description:

Sends a sequence of TCP packets to specific ports of a remote server. Used to enable SSH access or trigger port-knock based firewalls.

Usage:

```
python3 port_knocker.py <host> <port1> [port2] [port3] ...
```

Example

```
python3 port_knocker.py 192.168.1.100 7000 8000 9000 python3 port_knocker.py target.com 1234
```

Nota:

Requires at least one port. Timeout between knocks is 0.2 seconds.

```
port_knocker.main()
```

```
port_knocker.send_knock(host, port)
```

SYN Flooder

Master's in Computer Security Engineering Dynamic Programming Languages - SYN Flooder

Author: Martinho Caeiro (23917)

Description:

Sends TCP SYN packets to simulate a denial-of-service attack (SYN Flood/DoS). Uses Scapy for low-level packet crafting.

Usage:

```
python3 syn_flooder.py <target> <duration> [-port PORT] [-threads N]
```

Example

```
python3 syn_flooder.py 192.168.1.1 10 -port 80 -threads 4
```

Monitor:

```
sudo tcpdump -i eth0 -nn "tcp[tcpflags] & tcp-syn != 0"
```

`syn_flooder.flood_target(target, target_port, duration, num_threads)`
Send SYN flood with multiple threads.

`syn_flooder.init_scapy()`
Load Scapy only when executing the script.

`syn_flooder.main()`

`syn_flooder.syn_flood_worker(target, target_port, duration, packets_sent, lock)`
Worker thread that sends TCP SYN packets to a target.

UDP Flooder

Master's in Computer Security Engineering Dynamic Programming Languages - UDP Flooder

Author: Martinho Caeiro (23917)

Description:

Sends UDP packets to a target (UDP Flood attack).

Usage:

```
python3 udp_flooder.py [-target HOST] [-port PORT] [-duration SECONDS] [-threads N]
```

Example

```
python3 udp_flooder.py -target 192.168.1.1 -port 53 -duration 10
```

Test:

```
sudo tcpdump -i eth0 udp
```

`udp_flooder.flood_target(target, port, packet_size, duration, num_threads)`
Send UDP flood with multiple threads.

`udp_flooder.main()`

`udp_flooder.udp_flood_worker(target, port, packet_size, duration, packets_sent)`
Worker thread that sends UDP packets to a target.